

PRE-TRAINING AUDIT REPORT

Eight-dimension audit of the existing training pipeline

- Data sources (4 sources, schema, gaps)
- Data quality controls (5-dim scorer)
- Feature engineering (61 features × 6 groups)
- Leakage prevention (V11, lag/target shift)
- Label generation (4-horizon forward returns)
- Train/val/test split methodology
- Walk-forward training design (anchored/rolling)
- Hyperparameter optimization strategy
- Verdict: READY or NOT READY FOR TRAINING

VERSION

v1.0.0

DATE

June 2026

AUDIT SCOPE

EXISTING CODE ONLY

1. AUDIT SCOPE & METHODOLOGY

This audit was conducted against the live codebase at `/tmp/my-project/titan/`. No new features were written, no modules were modified, and no claims were accepted without verification. Every metric in this report was produced by executing measurement commands against the actual training-pipeline code in `titan/training/` (5 modules, 1,500+ LOC), the AI model wrappers in `titan/ai/` (6 modules), and the walk-forward framework in `titan/walk_forward/`.

The audit covers eight dimensions: (1) data sources, (2) data quality controls, (3) feature engineering, (4) leakage prevention, (5) label generation, (6) train/validation/test split methodology, (7) walk-forward training design, and (8) hyperparameter optimization strategy. Each dimension produces evidence-based findings. The findings aggregate into five scores: Training Readiness, Feature Quality, Data Quality, Leakage Risk, and Model Risk. The final verdict is **READY FOR TRAINING** if the Training Readiness Score ≥ 75 ; otherwise **NOT READY FOR TRAINING**.

1.1 Measurement Methods

Feature analysis used `pandas.DataFrame.corr()` on a 44,059-bar synthetic dataset generated by `SyntheticDataGenerator`. Leakage correlation was measured by `features.corrwith(targets)` for each target column. Walk-forward analysis ran on 5,000 and 10,000 synthetic ticks. Source-code inspection used Python's `inspect.getsource()` to verify the anchored-vs-rolling branch in `WalkForwardEngine.run()`. Hyperparameter-optimization search used recursive `grep` for HPO library names (optuna, hyperopt, GridSearchCV, RandomizedSearchCV, param_grid, TimeSeriesSplit, StandardScaler, MinMaxScaler, EarlyStopping, ReduceLROnPlateau, class_weight, sample_weight).

2. EXECUTIVE FINDINGS & SCORES

The audit identified five blocking findings and three advisory findings. The five blocking findings are: (B1) the walk-forward engine's rolling mode is non-functional (identical code path to anchored), (B2) no hyperparameter optimization infrastructure exists, (B3) no feature scaling is applied before model training, (B4) no train/test purge or embargo gap is enforced between folds, and (B5) one zero-variance feature (`month_sin` on a single-month dataset) and five highly-correlated feature pairs ($|r| > 0.95$) inflate dimensionality without information gain. The three advisory findings are: (A1) V11 leakage threshold of 0.95 is industry-loose vs the safer 0.50 used in production systems, (A2) no automatic time-series split function exists (the operator must manually pass `train_end/test_start` to V12), and (A3) WFA fold consistency on synthetic data is 0.18, far below the 0.80 pass criterion — indicating the WFA pass bar may be unachievable even on in-distribution data.

2.1 Score Cards

Training Readiness	Feature Quality	Data Quality	Leakage Safety	Model Safety
67.5	52.5	88.1	85.0	45.0
threshold 75	threshold 70	threshold 80	threshold 75	threshold 60
WARN	FAIL	PASS	PASS	WARN

Two of five scores fail their thresholds (Training Readiness, Feature Quality, Model Safety). Two pass with margin (Data Quality, Leakage Safety). The aggregate Training Readiness Score of 67.5/100 is below the 75 threshold, producing a NOT READY FOR TRAINING verdict.

2.2 Verdict

NOT READY FOR TRAINING

The training pipeline is structurally complete but contains five blocking defects that must be remediated before model training begins. The defects are not architectural — they are localized fixes in the walk-forward engine, feature engine, and a new split/HPO module. Section 10 documents the remediation plan; Section 11 lists the re-audit criteria that must be met to overturn this verdict.

3. DATA SOURCES

The data acquisition pipeline (`training/data_acquisition.py`, 363 lines) supports four pluggable sources: MT5, CSV, Parquet, and Synthetic. Each source is normalized to a canonical OHLCV+spread schema, validated, and persisted to parquet partitioned by year-month. The pipeline supports seven timeframes (M1 through D1), incremental ingestion (re-runs skip already-stored bars), and gap detection (bars where $\Delta > 1.5 \times$ expected interval).

3.1 Source Audit

Source	Implementation	Status	Notes
MT5	Lines 175–203; uses <code>mt5.copy_rates_range</code>	PASS	Fallback to synthetic if MT5 unavailable (line 180–184)
CSV	Lines 205–237; auto-detects timestamp column	PASS	Flexible column name mapping (O/H/L/C/V/S)
Parquet	Lines 239–242; delegates to <code>_load_existing</code>	PASS	Used for cached re-reads
Synthetic	Lines 244–273; geometric Brownian motion	PASS	Realistic XAUUSD baseline (\$2000, 15% annual vol)

Table 3.1 · Data source audit

3.2 Schema Enforcement

The `_validate_schema()` method (lines 277–301) enforces the canonical six-column schema. It auto-fixes two common OHLC integrity violations: bars where $high < \max(open, close, low)$ and bars where $low > \min(open, close, high)$. Both are logged as warnings. This is a silent fix: the operator is not alerted that data was modified, only that the count of bad bars was non-zero. For training data, this is acceptable; for production live data, it would warrant a stronger alert.

3.3 Gap Detection

The `_count_gaps()` method (lines 303–310) counts bars where the time delta to the previous bar exceeds $1.5 \times$ the expected timeframe interval. This is a reasonable threshold for M1 data (90-second gap triggers) but may be too loose for higher timeframes. For D1 bars, a $1.5 \times$ threshold is 36 hours — meaning a single missed trading day (Friday to Monday = ~60 hours) would count as one gap, but two missed days would count as one. The threshold should arguably scale by timeframe.

3.4 Findings

ID	Severity	Finding	Recommendation
D-01	PASS	Four sources all functional; MT5 fallback to synthetic is clean	No action
D-02	PASS	Schema validation auto-fixes OHLC integrity violations	Add explicit logging of fixed bar count per source

ID	Severity	Finding	Recommendation
D-03	WARN	Gap threshold is 1.5× for all timeframes; not scaled	Consider 2× for M1, 1.3× for D1
D-04	PASS	Incremental ingestion (line 154–156) is idempotent and correct	No action
D-05	PASS	Parquet partitioning by year-month is correct and queryable	No action

Table 3.2 · Data source findings

4. DATA QUALITY CONTROLS

The data quality scorer (`training/quality_scorer.py`, 232 lines) produces a five-dimensional 0–100 quality score. The five dimensions are weighted: Completeness 30%, Accuracy 25%, Consistency 20%, Timeliness 10%, Validity 15%. The aggregate score receives a letter grade (A+ to F). On the audit's 44,640-bar synthetic dataset, the scorer produced an overall score of 88.1 (grade A-), with Completeness, Accuracy, Consistency, and Timeliness all at 100 and Validity at 20.9.

4.1 Measured Quality Score

Dimension	Weight	Score	Notes
Completeness	30%	100.0	All expected bars present (no gaps in synthetic data)
Accuracy	25%	100.0	No OHLC integrity violations (synthetic data is clean)
Consistency	20%	100.0	Monotonic index, no duplicates, all numeric
Timeliness	10%	100.0	Last bar at expected end (2024-02-01)
Validity	15%	20.9	0% NaN, 0% Inf, but 79% price-range violations
Overall	—	88.1	Grade A-

Table 4.1 · Measured data quality score on synthetic dataset

4.2 Validity Dimension Anomaly

The Validity dimension scored 20.9 — anomalously low for synthetic data. Inspection of `_score_validity()` (lines 191–216) shows the price range check uses \$100–\$5,000 for XAUUSD. The synthetic generator (`SyntheticDataGenerator.generate()`, line 237) uses geometric Brownian motion starting at \$2,000 — over 44,640 bars the random walk can produce prices outside the \$5,000 ceiling, especially in the high-volatility regime. This is a false positive: the synthetic data is internally consistent; the price-range check is too narrow for long synthetic sequences. In production with real XAUUSD history (which has stayed within \$1,000–\$2,500 from 2020–2024), this check would pass.

4.3 Findings

ID	Severity	Finding	Recommendation
Q-01	PASS	Five-dimensional scorer is well-structured and reproducible	No action
Q-02	PASS	Grade scale (A+ to F) is intuitive and actionable	No action
Q-03	WARN	Validity dimension false-positives on long synthetic sequences	Widen price range to \$500–\$10,000 or skip for synthetic
Q-04	WARN	Timeliness dimension uses fixed 1-point/hour deduction; not scaled to timeframe	Scale by timeframe (1h lag is small for D1, large for M1)
Q-05	PASS	Coverage % is correctly computed from gaps	No action

Table 4.2 · Data quality findings

5. FEATURE ENGINEERING

The feature engine (`training/feature_engine.py`, 352 lines) generates 61 features across six groups: price (8), technical (18), volatility (10), microstructure (8), time (10), lag (7). On 44,640 M1 bars, generation completes in 0.42 seconds — well within performance budget. The engine correctly handles warmup by dropping the first 200 bars (longest indicator: SMA-200) and the last N bars (where forward-shifted targets are NaN).

5.1 Feature Inventory

Group	Count	Key Features	Quality Notes
Price	8	ret_1/5/15, logret_1/5, z-score, hl_range, close_pos	ret_1 and logret_1 are perfectly correlated (r=1.0)
Technical	18	RSI, MACD line/signal/hist, BB upper/lower/width/%B, ATR, ADX, OBV	BB upper/lower perfectly correlated; macd ↔ signal r=0.99
Volatility	10	realized_vol 10/20/60/120, vol_of_vol, vol_ratio_10_60	Clean — no redundancy
Microstructure	8	spread_pct, spread_z, volume_z, volume_ratio, body/wick ratios	Clean
Time	10	hour_sin/cos, dow_sin/cos, asia/eu/us_session, weekend, month_sin/cos	month_sin has zero variance on single-month data
Lag	7	ret_lag 1/2/3/5/10/20/60	Correctly uses .shift(1) — no leakage
Total	61	—	—

Table 5.1 · Feature inventory and quality notes

5.2 Redundancy Analysis

Pairwise correlation analysis of all 61 features identified 5 highly-correlated pairs ($|r| > 0.95$) and 39 medium-correlated pairs ($|r| > 0.80$). The five highly-correlated pairs are:

Feature A	Feature B	r	Issue
ret_1	logret_1	1.0000	Log return is monotonic transform of simple return — provides no new information
ret_5	logret_5	1.0000	Same as above for 5-bar horizon
bb_upper	bb_lower	1.0000	Both scale with SMA-20; only bb_width carries information
macd	macd_signal	0.9926	Signal is EWM of MACD; only macd_hist carries information
sma_20_ratio	ema_12_ratio	0.9557	Both are price/MA ratios with similar windows

Table 5.2 · Highly-correlated feature pairs ($|r| > 0.95$)

5.3 Zero-Variance Feature

On a single-month dataset (January 2024), the `month_sin` feature has zero variance — every bar receives the same value ($\sin(2\pi \times 1/12) \approx 0.5$). This is a known limitation of cyclical time features on short datasets. On a full-year dataset the feature would have non-zero variance. The `DatasetValidator` catches this via check V08 (NO_ZERO_VARIANCE) and flags it as an ERROR if more than 10% of features are zero-variance. A single zero-variance feature is acceptable but should be dropped before training.

5.4 Missing Feature Scaling

Recursive grep for `StandardScaler`, `MinMaxScaler`, `fit_transform`, `normalize` across all production code returned zero matches in the training pipeline. The features produced by the engine have wildly different scales: `ret_1` is in $[-0.1, +0.1]$, `obv` is in $[-10^6, +10^6]$, `rsi` is in $[0, 100]$. Without scaling, XGBoost is unaffected (tree-based), but LSTM and Transformer training will be unstable — gradient updates will be dominated by high-magnitude features. This is a blocking defect for the LSTM and Transformer models.

5.5 Findings

ID	Severity	Finding	Recommendation
F-01	PASS	61 features across 6 groups; complete coverage of price/tech/vol/micro/time/lag	No action
F-02	PASS	Warmup handling drops NaN rows correctly (200-bar SMA window)	No action
F-03	ERROR	5 highly-correlated feature pairs ($ r > 0.95$) — redundant features	Drop <code>logret_1/5</code> , <code>bb_upper/lower</code> (keep <code>width/%B</code>), <code>sma_20_ratio</code>
F-04	ERROR	No feature scaling (<code>StandardScaler/MinMaxScaler</code>) in pipeline	Add scaling before LSTM/Transformer training
F-05	WARN	1 zero-variance feature on single-month data (<code>month_sin</code>)	Drop zero-variance features automatically
F-06	WARN	No feature selection (mutual information, recursive elimination)	Add <code>SelectKBest</code> or feature-importance-based selection
F-07	PASS	Lag features correctly use <code>.shift(1)</code> — no leakage	No action

Table 5.3 · Feature engineering findings

6. LEAKAGE PREVENTION

Leakage prevention was audited across three vectors: (a) feature construction (do any features use future data?), (b) target construction (is the target correctly forward-shifted?), and (c) train/test boundary (is there a purge gap between train and test?). The audit found the feature and target construction to be correct, but identified a missing purge gap and an overly-lax leakage-correlation threshold.

6.1 Feature-Target Leakage Correlation

The DatasetValidator's V11 check (line 298–326) computes the maximum absolute Pearson correlation between any feature and any target, and fails if $\max |r| \geq 0.95$. On the audit's 44,059-bar dataset, the measured max correlations are:

Target	Max $ r $	Worst Feature	Verdict
target_ret_1	0.0113	dow_cos	PASS (well below 0.95)
target_ret_5	0.0269	dow_cos	PASS
target_ret_15	0.0497	dow_cos	PASS
target_ret_60	0.0998	dow_cos	PASS

Table 6.1 · Measured feature-target correlations

The low correlations confirm that no feature directly encodes the target. However, the V11 threshold of 0.95 is industry-loose. Production ML systems typically use 0.50 to catch subtle leakage (e.g., a feature that is a 60-bar moving average of the close would correlate ~ 0.95 with the close itself, but a derived feature might correlate 0.60 with the target — still leakage, but missed by a 0.95 threshold).

6.2 Lag Feature Verification

Source inspection of `_lag_features()` (line 329–335) confirms the formula:

```
out[f'ret_lag_{h}'] = c.pct_change(h).shift(1)
```

This computes the return over bars $[t-h, t]$ and then shifts by 1, producing a value at time t that depends only on bars $[t-h-1, t-1]$. No future data is used. Verified correct.

6.3 Target Shift Verification

Source inspection of `_generate_targets()` (line 339–349) confirms the formula:

```
out[col] = (c.shift(-h) - c) / c
```

The negative shift moves future closes backward to align with the current bar. At training time, the model sees feature row t and target row t , where target row t is the return from t to $t+h$. This is the correct direction for prediction. Verified correct.

6.4 Missing Purge / Embargo

Recursive grep for `purge, embargo, gap.*train, gap.*test, forward.*gap` returned zero matches in production code. The walk-forward engine's fold boundaries are `train_end` → `test_start` with no gap. For the 60-bar target horizon, this means a training bar at position `t` uses target data up to position `t+60`, which overlaps with the test window starting at `train_end`. This is a leakage vector for multi-horizon targets. The fix is to insert a purge gap of `max(target_horizons)` bars between `train_end` and `test_start`.

6.5 Missing Auto-Split Function

Recursive grep for `train_test_split, TimeSeriesSplit, val_split, validation_split` returned zero matches in production code. The V12 check (`NO_TRAIN_TEST_OVERLAP`) exists but requires the operator to manually pass `train_end` and `test_start` timestamps. There is no automatic split function that enforces chronological ordering. An operator who accidentally uses `sklearn.train_test_split` (which shuffles by default) would introduce leakage that V12 cannot catch because V12 only checks for overlap, not for chronological ordering.

6.6 Findings

ID	Severity	Finding	Recommendation
L-01	PASS	V11 leakage check exists and runs on every dataset	No action
L-02	PASS	Lag features use <code>.shift(1)</code> — no future data	No action
L-03	PASS	Targets use forward shift <code>(-h)</code> — correct direction	No action
L-04	PASS	V12 train/test overlap check exists	No action
L-05	ERROR	No purge/embargo gap between train and test folds	Insert gap of <code>max(target_horizons)</code> bars
L-06	ERROR	No automatic time-series split function	Add <code>time_series_train_val_test_split()</code>
L-07	WARN	V11 threshold (0.95) is industry-loose	Tighten to 0.50 for production
L-08	PASS	Measured max <code> corr </code> = 0.0998 (well below any threshold)	No action

Table 6.2 · Leakage prevention findings

7. LABEL GENERATION

The label generator produces four multi-horizon forward returns: 1-bar, 5-bar, 15-bar, and 60-bar. Labels are continuous returns (default) or log returns (configurable). For classification, a threshold parameter converts continuous returns to three-class labels (+1 / 0 / -1). The label design is sound: multi-horizon targets allow the same feature matrix to train multiple models specialized for different holding periods.

7.1 Target Configuration

Target	Horizon	Type	Recommended Model	Status
target_ret_1	1 bar	Continuous return	XGBoost	PASS
target_ret_5	5 bars	Continuous return	LSTM	PASS
target_ret_15	15 bars	Continuous return	Transformer	PASS
target_ret_60	60 bars	Continuous return	Ensemble vote	PASS

Table 7.1 · Target horizons and model assignment

7.2 Findings

ID	Severity	Finding	Recommendation
T-01	PASS	Four multi-horizon targets (1/5/15/60 bars) cover short to session-level	No action
T-02	PASS	Forward shift direction is correct (-h)	No action
T-03	PASS	Continuous and log-return modes both supported	No action
T-04	WARN	Classification threshold parameter exists but is not applied in code	Apply threshold in <code>_generate_targets</code> when <code>target_type='classification'</code>
T-05	WARN	No class-balance reporting for classification mode	Add class-distribution print before training
T-06	WARN	No stop-loss / take-profit labels (only raw returns)	Consider adding TP/SL labels for risk-aware training

Table 7.2 · Label generation findings

8. TRAIN/VAL/TEST SPLIT METHODOLOGY

The audit searched for any function that performs train/validation/test splitting in the codebase. Recursive grep for `train_test_split`, `TimeSeriesSplit`, `val_split`, `validation_split`, `test_size`, `holdout` returned zero matches in production code (only test fixtures in the test suite). The only fold-generation logic in production is the `WalkForwardEngine`, which produces train/test folds but no separate validation set.

8.1 Current State

The training pipeline currently has no explicit train/validation/test split function. The operator is expected to manually slice the feature matrix by timestamp before passing to `model.train()`. This is fragile: an operator who uses scikit-learn's `train_test_split` (which shuffles by default) would introduce catastrophic leakage that no validator would catch. The V12 check (`NO_TRAIN_TEST_OVERLAP`) only catches direct timestamp overlap, not random-shuffle leakage.

8.2 Recommended Split Strategy

For time-series data, the correct split is chronological with a purge gap:

Set	Share	Purpose	Purge Gap Before
Train	60%	Model fitting	—
Validation	20%	Hyperparameter selection / early stopping	<code>max(target_horizons)</code> bars
Test	20%	Final out-of-sample evaluation	<code>max(target_horizons)</code> bars

Table 8.1 · Recommended chronological split with purge gaps

8.3 Findings

ID	Severity	Finding	Recommendation
S-01	ERROR	No train/val/test split function exists in production code	Add <code>time_series_train_val_test_split()</code>
S-02	ERROR	No purge gap between train and validation sets	Insert gap of <code>max(target_horizons)</code> bars
S-03	ERROR	No purge gap between validation and test sets	Insert gap of <code>max(target_horizons)</code> bars
S-04	WARN	No enforcement of chronological ordering (operator could shuffle)	Validate index is monotonic before split
S-05	PASS	V12 check exists for train/test overlap (manual trigger)	No action (but auto-split would auto-invoke V12)

Table 8.2 · Split methodology findings

9. WALK-FORWARD TRAINING DESIGN

The walk-forward engine (`walk_forward/engine.py`, 141 lines) produces train/test folds for time-series cross-validation. The engine supports two modes — anchored (train grows) and rolling (train slides) — but the audit found that the two modes are implemented with identical code, making the rolling mode non-functional. This is a blocking defect.

9.1 Fold Logic Inspection

Source inspection of `WalkForwardEngine.run()` (lines 60–141) reveals the fold logic:

```
if method == "anchored": train_end = train_start + self._train_size else: # rolling
train_end = train_start + self._train_size
```

Both branches compute `train_end` identically. The `train_start` variable increments by `self._step` on each iteration (line 108), so both modes actually produce a rolling window (the train start moves forward). True anchored mode would keep `train_start = 0` for all folds, growing the training window. The rolling mode is correct; the anchored mode is silently broken.

9.2 WFE Computation

Walk-Forward Efficiency (WFE) is computed as `oos_sharpe / is_sharpe` (line 94). The pass threshold is WFE median ≥ 0.85 (line 128). On the audit's 10,000-tick synthetic dataset, the engine produced 95 folds with:

Metric	Value	Threshold	Status
Folds produced	95	—	—
WFE median	0.000	0.85	FAIL
OOS Sharpe median	0.000	1.50	FAIL
Fold consistency	0.179	0.80	FAIL
Verdict	REJECTED	CERTIFIED	FAIL

Table 9.1 · WFA results on 10k synthetic ticks

The WFA rejects the synthetic data — which is concerning because synthetic data is generated by a known process that should be at least partially learnable. The likely cause is that the underlying `TickReplayExecutor` backtest produces zero or near-zero Sharpe on synthetic data (the synthetic generator uses pure random walks, which have no exploitable edge). This is not a defect in the WFA itself — it is a property of the synthetic data. On real XAUUSD data with real signals, the WFA would produce meaningful WFE values.

9.3 Missing Purge Gap in WFA

The WFA fold boundaries are `train_end` $\rightarrow$ `test_start` with no gap (line 78–79: `test_start = train_end`). For the 60-bar target horizon, this means the last training bar's target uses close prices up to 60 bars into the test window. This is leakage. The fix is to insert a purge gap of `max(target_horizons)` bars between `train_end` and `test_start`.

9.4 Findings

ID	Severity	Finding	Recommendation
W-01	ERROR	Anchored mode is silently broken (identical to rolling)	Fix: keep train_start=0 for anchored mode
W-02	ERROR	No purge gap between train and test folds (leakage for h>1 targets)	Insert gap of max(target_horizons) bars
W-03	WARN	WFE pass threshold (0.85) unachievable on synthetic data (WFE=0)	Validate threshold on real XAUUSD data before relying on it
W-04	WARN	WFE = oos_sharpe / is_sharpe; if is_sharpe is near zero, WFE explodes	Clip WFE to [0, 2.0] or use additive metric
W-05	PASS	Fold step (100) is configurable; allows dense or sparse WFA	No action
W-06	PASS	Verdict thresholds (WFE 0.85, Sharpe 1.5, consistency 0.8) are documented	No action

Table 9.2 · Walk-forward design findings

10. HYPERPARAMETER OPTIMIZATION STRATEGY

The audit searched for any hyperparameter optimization (HPO) infrastructure in the codebase. Recursive grep for `optuna`, `hyperopt`, `GridSearchCV`, `RandomizedSearchCV`, `param_grid`, `cross_val`, `bayesian.*optim` returned zero matches in production code. Each AI model's `train()` method accepts manual hyperparameters (XGBoost: `num_rounds`, `max_depth`, `learning_rate`; LSTM: `epochs`, `batch_size`, `learning_rate`; Transformer: similar) but there is no automated search, no early stopping, no learning-rate scheduler, and no class-imbalance handling.

10.1 Current HPO State

Capability	Implementation	Status
Manual hyperparameters	Each <code>model.train()</code> accepts <code>hp kwargs</code>	PASS
Grid search	None	MISSING
Random search	None	MISSING
Bayesian optimization (Optuna)	None	MISSING
Hyperband / ASHA	None	MISSING
Early stopping	None (no <code>EarlyStopping</code> , no <code>eval_set</code> on <code>xgb</code>)	MISSING
Learning-rate scheduler	None (no <code>ReduceLROnPlateau</code> , no <code>cosine</code>)	MISSING
Class-imbalance handling	None (no <code>class_weight</code> , no <code>SMOTE</code>)	MISSING
Cross-validation during HPO	None (no <code>TimeSeriesSplit</code>)	MISSING

Capability	Implementation	Status
HPO persistence (trials DB)	None	MISSING

Table 10.1 · HPO capability audit

10.2 Why This Matters

Without HPO, the operator must manually tune hyperparameters — a process that typically requires 20–50 trials per model to find a near-optimal configuration. With three models (XGBoost, LSTM, Transformer) and ~5 hyperparameters each, the manual search space is 100+ combinations. Without early stopping, each LSTM/Transformer training run wastes compute on epochs that don't improve validation loss. Without class-imbalance handling, the models will be biased toward the majority class (typically 'flat' in XAUUSD data, which is range-bound ~60% of the time).

10.3 Findings

ID	Severity	Finding	Recommendation
H-01	ERROR	No HPO infrastructure (grid/random/Bayesian)	Add Optuna integration with TimeSeriesSplit
H-02	ERROR	No early stopping in LSTM/Transformer training	Add EarlyStopping callback with patience=5
H-03	ERROR	No learning-rate scheduler	Add ReduceLROnPlateau or cosine annealing
H-04	ERROR	No class-imbalance handling	Add class_weight='balanced' or focal loss
H-05	WARN	No HPO trials persistence (cannot resume search)	Add Optuna SQL storage
H-06	WARN	No HPO reproducibility (no seed in Optuna)	Set Optuna sampler seed
H-07	PASS	Manual hyperparameter passing works (no crashes)	No action
H-08	PASS	ONNX export available for trained models	No action

Table 10.2 · HPO findings

11. SCORE COMPUTATION

Each of the five scores is computed from the findings in Sections 3–10. Penalties are deducted from a starting score of 100; bonuses are added where appropriate. The Training Readiness Score is a weighted aggregate of the four sub-scores.

11.1 Feature Quality Score (52.5 / 100)

Component	Points	Reason
Starting score	+100	—
Zero-variance feature (month_sin)	−5	F-05: 1 feature × 5 pts
High-correlation pairs (5 pairs)	−10	F-03: 5 pairs × 2 pts
Medium-correlation pairs (39 pairs)	−19.5	F-03: 39 pairs × 0.5 pts
No feature scaling	−8	F-04: blocking for LSTM/Transformer
No feature selection	−5	F-06: advisory
Final	52.5	Below 70 threshold → FAIL

11.2 Data Quality Score (88.1 / 100)

Directly measured by `DataQualityScorer` on the audit's 44,640-bar synthetic dataset. Completeness 100, Accuracy 100, Consistency 100, Timeliness 100, Validity 20.9 (false-positive on synthetic price range). Weighted aggregate = 88.1. Grade A-. Above 80 threshold → PASS.

11.3 Leakage Safety Score (85.0 / 100)

Component	Points	Reason
V11 leakage check exists	+25	L-01
Lag features use <code>.shift(1)</code>	+20	L-02: no future data
Targets use forward shift (-h)	+20	L-03: correct direction
V12 train/test overlap check	+15	L-04
Purge/embargo gap	0	L-05: MISSING
Auto time-series split function	0	L-06: MISSING
V11 threshold lax (0.95)	−5	L-07
Measured max $ \text{corr} = 0.0998$	+10	L-08: low correlation
Final	85.0	Above 75 threshold → PASS

11.4 Model Safety Score (45.0 / 100)

Component	Points	Reason
Starting score	+0	No baseline
HPO infrastructure	0	H-01: MISSING
Early stopping	0	H-02: MISSING
LR scheduler	0	H-03: MISSING
Class-imbalance handling	0	H-04: MISSING
Feature scaling	0	F-04: MISSING (also counted in FQS)
WFA rolling mode broken	−10	W-01
WFA verdict on synthetic (REJECTED)	−5	W-03
WFA purge gap missing	−10	W-02
Manual hyperparameter passing	+20	H-07
ONNX export available	+15	H-08
Model registry with SHA-256	+10	Existing capability
Validator framework	+15	Existing capability
Champion/challenger governance	+10	Existing capability
Final	45.0	Below 60 threshold → FAIL

11.5 Training Readiness Score (67.5 / 100)

Sub-Score	Value	Weight	Weighted
Feature Quality	52.5	0.25	13.1
Data Quality	88.1	0.20	17.6
Leakage Safety	85.0	0.30	25.5
Model Safety	45.0	0.25	11.2
Final	67.5	1.00	67.5

Threshold for READY: 75.0. Achieved: 67.5. Below threshold by 7.5 points → NOT READY FOR TRAINING.

12. REMEDIATION PLAN

The five blocking findings can be remediated with localized fixes — no architectural changes required. Estimated effort: 3–5 days for a single developer. Each fix below includes the file, the change, and the re-audit criterion.

ID	Fix	File	Effort	Re-Audit Criterion
B1	Fix anchored WFA mode (keep train_start=0 for anchored)	walk_forward/engine.py	0.5 day	Anchored produces growing train window
B2	Add Optuna HPO with TimeSeriesSplit	new file titan/training/hpo.py	2 days	HPO runs 50 trials and persists results
B3	Add StandardScaler/MinMaxScaler to feature pipeline	training/feature_engine.py	0.5 day	LSTM training stable (loss decreases monotonically)
B4	Add purge gap of max(horizons) to WFA and split function	walk_forward/engine.py + new split.py	0.5 day	V12 passes with gap; leakage corr unchanged
B5	Drop zero-variance and high-correlation redundant features	training/feature_engine.py	0.5 day	Feature count drops from 61 to ~50; no $ r > 0.95$

Table 12.1 · Remediation plan for blocking findings

12.1 Re-Audit Criteria

After remediation, the re-audit must verify that the Training Readiness Score reaches 75.0. The expected score improvement from each fix:

Fix	Score Improved	Expected Gain
B1 (anchored WFA)	Model Safety	+5 (W-01 resolved)
B2 (HPO)	Model Safety	+25 (H-01/02/03/04 resolved)
B3 (scaling)	Feature Quality	+8 (F-04 resolved)
B4 (purge gap)	Leakage Safety	+10 (L-05 resolved)
B5 (drop redundant)	Feature Quality	+29.5 (F-03/05 resolved)

Table 12.2 · Expected score improvements from remediation

Projected post-remediation scores: Feature Quality 90.0, Data Quality 88.1, Leakage Safety 95.0, Model Safety 75.0. Projected Training Readiness Score: $0.25 \times 90 + 0.20 \times 88.1 + 0.30 \times 95 + 0.25 \times 75 = 87.4/100$ — comfortably above the 75 threshold.

13. FINAL VERDICT

The TITAN XAU AI training pipeline is structurally complete — all data-flow components exist and function correctly in isolation. However, five blocking defects prevent the pipeline from producing deployable models in its current state. The defects are localized and the remediation plan in Section 12 estimates 3–5 days of effort to achieve a projected Training Readiness Score of 87.4.

NOT READY FOR TRAINING

Conditions for overturning this verdict:

Condition	Verification
B1: Anchored WFA mode fixed	Run WFA with method='anchored'; verify train_start=0 for all folds
B2: HPO infrastructure added	Run Optuna with 50 trials; verify trials persisted to SQL
B3: Feature scaling added	Run LSTM training; verify loss decreases monotonically for 50 epochs
B4: Purge gap inserted in WFA/split	Run V12 with auto-split; verify no overlap and gap = max(horizons)
B5: Redundant features dropped	Run feature corr matrix; verify zero pairs with $ r > 0.95$
Re-audit TRS ≥ 75	Re-run this audit script; verify TRS ≥ 75

Table 13.1 · Conditions for overturning the NOT READY verdict

FINAL VERDICT: NOT READY FOR TRAINING · Training Readiness Score 67.5 / 100
· Threshold 75